

PROGRAMMIERUNG I

Kapitel 3: Rekursion, Datenstrukturen und Fehler

Arrays

Instanziierung und Zugriff

Aktuell kann die **Zoo**-Klasse in ihren Variablen maximal vier Tiere aufnehmen. Ein richtiger Zoo hat aber natürlich weitaus mehr Tiere.

Dafür die **Zoo**-Klasse um unzählige weitere Instanzvariablen zu erweitern ist nicht praktikabel. Stattdessen kann eine neue Datenstruktur verwendet werden: **Arrays**.

Ein Array ist ein Container für eine feste Anzahl an Objekten desselben Typs, etwa vergleichbar mit einem Regal für typgleiche Objekte. Einmal festgelegt, kann die Größe eines Arrays nicht mehr verändert werden.

Arrays werden folgendermaßen deklariert und instanziiert (zu beachten sind die eckigen Klammern):

Syntax (Deklaration eines Arrays):
`Type[] arrayVar;`

Syntax (Deklaration und Instanziierung eines Arrays der Größe `size`):
`Type[] arrayVar = new Type[size];`

Die Elemente innerhalb eines Arrays werden über deren Index-Position referenziert:

Syntax (Ablegen des Wertes `someVariable` an der Position `index`):
`arrayVar[index] = someVariable;`

Syntax (Lesen des Wertes `someVariable` von der Position `index`):
`someVariable = arrayVar[index];`

Die erste Index-Position ist immer 0, die letzte Index-Position ist dementsprechend Größe minus 1.

Arrays können für Klassen (z.B. `String[]` oder `Duck[]`) als auch für primitive Datentypen (z.B. `int[]` oder `double[]`) angelegt werden. Wenn an einer Position des Arrays noch kein konkreter Wert abgelegt wurde, so liefert ein Lesezugriff auf diese Position den Wert `null` zurück (bzw. 0 bei primitiven Zahl-Datentypen).

Iteration

Um dieselbe Aktion für jedes Element innerhalb eines Arrays auszuführen, kann über das Array **iteriert** werden.

Hierfür kann die bekannte **for**-Schleife verwendet werden:

```
public class Zoo {

    // Deklaration und Instanziierung des Arrays.
    Duck[] ducks = new Duck[100];
    // Um das Beispiel kurz zu halten, wurden die anderen Tierarten temporär entfernt.

    public void addAllAnimals() {
        ducks[0] = new Duck("Freddy", 2);
        ducks[1] = new Duck("Tina", 1);
        ducks[2] = new Duck("Klara", 2);
        ducks[3] = new Duck("Johnny", 2);
        ducks[4] = new Duck("Mikey", 3);
        ducks[5] = new Duck("Mindy", 3);
        ducks[6] = new Duck("Lisa", 5);
        ducks[7] = new Duck("Richard", 6);
    }

    public void listAllAnimals() {
        // Hier wird mittels einer klassischen for-Schleife über alle Elemente des Duck-Arrays iteriert.
        // Falls an der jeweiligen Position ein Objekt hinterlegt wurde, wird dessen toString-Methode aufgerufen.

        for (int index=0; index < ducks.length; index++) {
            Duck duck = ducks[index];
            if (duck != null) {
                System.out.println(duck.toString());
            }
        }
    }
}
```

Eine modernere (und kürzere) Variante der Iteration ist die **ForEach-Schleife**:

```
Syntax:  
for (ElementType currentElement : arrayVariable) {  
    // in jeder Iteration referenziert currentElement auf das jeweils aktuelle Array-Element.  
}
```

Beispiel:

```
public class Zoo {  
  
    // Deklaration und Instanziierung des Arrays.  
    Duck[] ducks = new Duck[100];  
    // Um das Beispiel kurz zu halten, wurden die anderen Tierarten temporär entfernt.  
  
    public void addAllAnimals() {  
        ducks[0] = new Duck("Freddy", 2);  
        ducks[1] = new Duck("Tina", 1);  
        ducks[2] = new Duck("Klara", 2);  
        ducks[3] = new Duck("Johnny", 2);  
        ducks[4] = new Duck("Mikey", 3);  
        ducks[5] = new Duck("Mindy", 3);  
        ducks[6] = new Duck("Lisa", 5);  
        ducks[7] = new Duck("Richard", 6);  
    }  
  
    public void listAllAnimals() {  
        // Hier wird mittels einer foreach-Schleife über alle Elemente des Duck-Arrays iteriert.  
        // Falls an der jeweiligen Position ein Objekt hinterlegt wurde, wird dessen toString-Methode aufgerufen.  
  
        for (Duck duck : ducks) {  
            if (duck != null) {  
                System.out.println(duck.toString());  
            }  
        }  
    }  
}
```

Die Iteration erfolgt über alle Array-Indizes – egal ob dort der Wert `null` vorliegt.

Erweiterte Objektorientierung

Polymorphie

Der Zoo fasst jetzt eine ordentliche Anzahl an **Ducks**, **Ostrichs** und **Hummingbirds**. Alle Objekte werden in einem von insgesamt 3 tierart-spezifischen Arrays gespeichert. Um eine komplett neue Tierart hinzuzufügen, müsste ein weiteres Array erzeugt werden.

Da größere Zoos aber oft mehrere hundert verschiedene Tierarten enthalten, ist das Hinzufügen von hunderten Arrays keine praktikable Lösung.

Da Java **Polymorphie** unterstützt, können alle **Ducks**, **Ostrichs** und **Hummingbirds** einfach in demselben **Animal**-Array (oder **Bird**-Array) gespeichert werden, denn jedes einzelne dieser Objekte IST ja auch ein **Animal** und **Bird** (aber nicht jedes ist ein **FlyingBird**), z.B.:

```
Animal[] animals = new Animal[1000];  
animals[0] = new Duck("Freddy", 2);
```

Polymorphie bedeutet „Vielgestaltigkeit“, d.h. dass für ein und dasselbe Objekt mehrere dieser Aussagen gleichzeitig gelten können „Das Objekt *IST EIN* ...“ – z.B.:

EIN Duck IST EIN Animal

EIN Duck IST EIN Bird

EIN Duck IST EIN FlyingBird

Im genannten Beispiel erstreckt sich die Polymorphie auf alle Überklassen (von **Duck**). Tatsächlich erstreckt sich Polymorphie zusätzlich auch auf implementierte Interfaces – dazu später mehr. Polymorphie funktioniert nur bei Objekt-Datentypen (nicht bei primitiven).

Polymorphie bedeutet:

Variablen können nicht nur Objekte des angegebenen Typs fassen, sondern auch Objekte eines entsprechenden Subtyps (Unterklasse oder Interface):

```
Bird bird = new Duck("Freddy", 2);
```

Methoden akzeptieren als Übergabeparameter nicht nur die angegebenen Typen, sondern auch alle entsprechenden Subtypen:

```
public void run() {  
    Duck duck = new Duck("Freddy", 2);  
    eatAndFlapWings(duck);  
}  
  
public void eatAndFlapWings(Bird bird) {  
    bird.eat();  
    bird.flapWings(3);  
}
```

Rückgabewerte von Methoden dürfen dem angegebenen Typen entsprechen oder einem Subtypen:

```
public Animal createRandomAnimal() {  
    if (Math.random() > 0.5) {  
        return new Duck("Nick", 5);  
    } else {  
        return new Ostrich("Anna", 9);  
    }  
}
```

Verwendung der Java-Basisklasse **Math**. Für Details dazu, bitte das entsprechende [JavaDoc](#) ansehen.

Es wird erst zur Laufzeit entschieden, welche Methode aufgerufen wird. Dabei wird diejenige Methode ausgeführt, die in der Klasse am niedrigsten im Vererbungsbaum implementiert ist. Wenn beispielsweise sowohl **Animal** als auch **Duck** eine **toString()** Methode definieren, dann wird hier die Methode der **Duck**-Klasse aufgerufen (welche die Methode von **Animal** überschreibt):

```
Animal animal = new Duck("Freddy", 2);  
System.out.println(animal.toString());
```

```
public class Zoo {  
  
    // Deklaration und Instanziierung des Animal-Arrays.  
    // Dieses kann Objekte von allen verschiedenen Unterklassen von Animal enthalten.  
    // Dieses Array könnte auch ein Bird-Array sein.  
    // Wäre es aber ein FlyingBird-Array, so könnte es keine Ostrichs aufnehmen.  
    Animal[] animals = new Animal[1000];  
  
    public void addAllAnimals() {  
        // Das Array ist vom Typ "Animal",  
        // aber es kann auch Objekte fassen die von "Animal" erben.  
        animals[0] = new Duck("Freddy", 2);  
        animals[1] = new Duck("Tina", 1);  
        animals[2] = new Duck("Klara", 2);  
        animals[3] = new Ostrich("Carlos", 12);  
        animals[4] = new Ostrich("Nina", 10);  
        animals[5] = new Hummingbird("Cindy", 2);  
        animals[6] = new Hummingbird("Tobi", 2);  
    }  
  
    public void listAllAnimals() {  
        // Hier wird mittels einer foreach-Schleife über alle Elemente des Animal-Arrays iteriert.  
        // Falls an der jeweiligen Position ein Objekt hinterlegt wurde, wird dessen toString-Methode aufgerufen.  
        // Es können nur Methoden aufgerufen werden, die jedes Animal-Objekt garantiert besitzt,  
        // d.h. animal.fly() wäre nicht erlaubt, da nicht jedes Animal-Objekt diese Methode anbietet.  
  
        for (Animal animal : animals) {  
            if (animal != null) {  
                System.out.println(animal.toString());  
            }  
        }  
    }  
}
```

Um zur Laufzeit den tatsächlichen Typen eines beliebigen Objekts ermitteln zu können, gibt es folgende Möglichkeiten:

1. Jedes Objekt besitzt die Methode `getClass()`, mittels welcher zur Laufzeit der Name der tatsächlichen Klasse ermittelt werden kann. Damit können wir z.B. die `toString()` Methode einmal an zentraler Stelle (`Animal`) definieren und dadurch Redundanz vermeiden:

```
public class Animal {  
    private int age;  
    private String name;  
  
    public String toString() {  
        return "I am " + name + " the " + this.getClass().getSimpleName() + " and my age is " + age + ".";  
    }  
}
```

2. In Verzweigungen kann das Keyword `instanceof` verwendet werden:

```
public class Zoo {  
    Animal[] animals = new Animal[1000];  
  
    void letAllBirdsFly() {  
        // Iterieren über alle Animals:  
        for (Animal currentAnimal : animals) {  
            // Prüfung ob currentAnimal ein FlyingBird ist:  
            if (currentAnimal instanceof FlyingBird) {  
                // Explizites Casting von Animal nach FlyingBird:  
                FlyingBird currentFlyingBird = (FlyingBird) currentAnimal;  
                // Jetzt können wir mit der currentFlyingBird Variable arbeiten:  
                currentFlyingBird.sayName();  
                currentFlyingBird.flapWings(3);  
                currentFlyingBird.fly();  
            }  
        }  
    }  
}
```

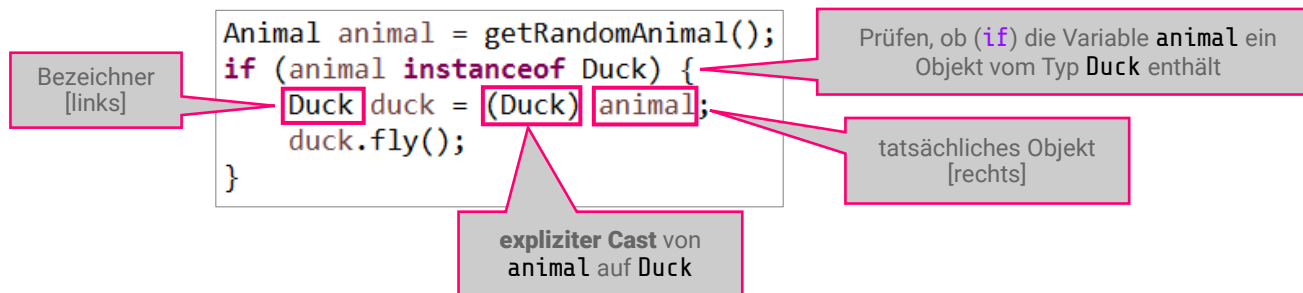

Wenn der Typ des tatsächlichen Objekts (rechts) garantiert gleich dem Typ des Bezeichners ist (einfachster Fall), dann handelt es sich nicht um Polymorphie und es ist keine Typumwandlung erforderlich:



Wenn das tatsächliche Objekt (rechts) garantiert ein Subtyp des Bezeichners ist („[rechts] ist garantiert ein [links]“), dann führt Java die Umwandlung des Typen (das **Casting**) automatisch (implizit) durch, da hierbei kein Risiko für **Casting-Fehler** besteht:



Wenn nicht garantiert werden kann, dass das tatsächliche Objekt (rechts) ein Subtyp des Bezeichners ist („[rechts] ist möglicherweise kein [links]“), dann kann bei Bedarf eine manuelle (**explizite**) Typumwandlung durchgeführt werden. Da hierbei das Risiko für Casting-Fehler besteht, sollte vor dem Casting mittels **instanceof** sichergestellt werden, dass „[rechts] ist in diesem Fall ein [links]“:



Interfaces

Bisher haben wir innerhalb der Animals zwischen **Birds** und **FlyingBirds** unterschieden. Neben der Flugfähigkeit gibt es aber noch weitere Aspekte, anhand derer Vögel unterschieden werden können und die für den Zoo relevant sind, z.B.:



MigratoryBirds (Zugvögel)

vs. **ResidentialBirds** (Standvögel)

WaterBirds (Wasservögel)

vs. **FieldBirds** (Feldvögel)

vs. **ForestBirds** (Waldvögel)

HerbivoreBirds (Pflanzenfresser)

vs. **CarnivoreBirds** (Fleischfresser)

vs. **OmnivoreBirds** (Allesfresser)

Um dies mit Vererbung abzubilden, müssten wir eine Menge weiterer Unterklassen von **Bird** erstellen, z.B.:

- **OmnivoreMigratoryFlyingWaterBird** (z.B. für Duck)
- **HerbivoreResidentialNonFlyingFieldBird** (z.B. für Ostrich)
- **HerbivoreResidentialFlyingForestBird** (z.B. für Hummingbird)

Die Einführung weiterer Tierarten oder Aspekte würde sicherlich zu einer Explosion der verschiedenen Zwischen-Klassen führen. Diese Art der ausschließlich hierarchischen Gruppierung von Klassen ist daher nicht zielführend.

Dieses Problem lässt sich durch die Verwendung von Interfaces lösen.

Ähnlich zu Klassen, können Interfaces anzeigen, was ein Objekt kann. Ein Interface ist eine normale Java Class-Datei, welche jedoch ausschließlich eine beliebige Anzahl an Methodensignaturen enthält. D.h. ein Interface hat niemals Methodenrumpfe, Schleifen oder Verzweigungen:

Syntax (Definition eines Interfaces welches zwei Methoden definiert):

```
public interface MyInterface {  
    void myMethod();  
    int myOtherMethod(long myParameter);  
}
```

keine Methodenrumpfe!

Ein Interface definiert, WAS ein Objekt können muss.
Eine Klasse definiert, WIE genau die geforderte Funktionalität implementiert ist.

Konkrete Java-Klassen können (mehrere) Interfaces implementieren und damit anzeigen dass sie alle im Interface definierten Funktionalitäten bereitstellen:

Syntax:

```
public class MyClass implements MyInterface, OtherInterface
```

Wenn eine konkrete Klasse ein Interface implementiert, muss sie zwingend alle im Interface angegebenen Methoden implementieren (d.h. die zu den Methodensignaturen gehörigen Methodenrumpfe ausprogrammieren).

Wenn eine Klasse ein Interface implementiert, dann nimmt sie den Typen des implementierten Interfaces an (*IST EIN*). Somit kann eine Klasse mehrere Typen gleichzeitig annehmen (Polymorphie).

Es ist auch üblich, Interfaces zu definieren welche überhaupt keine Methoden definieren. Diese werden als **Marker-Interfaces** bezeichnet.

Ein Interface kann von anderen Interfaces erben – sogar von mehreren gleichzeitig:

Syntax:

```
public interface MyInterface extends OtherInterface, OneMoreInterface
```

Es ist möglich, aber unüblich, in Interfaces Variablen zu deklarieren. Diese werden automatisch in Klassen-Konstanten (**static final**) umgewandelt.

Häufig wird bei der Entwicklung von Software zuerst definiert, *WAS* die verschiedenen Programmkomponenten tun sollen, d.h. die entsprechenden Interfaces werden definiert. Erst danach wird in den konkreten Klassen implementiert, *WIE* genau diese Funktionalitäten umgesetzt werden.

Eine **API (Application Programming Interface)** definiert ein oder mehrere Interfaces mit den zugehörigen Methodensignaturen. Häufig sind diese besonders umfangreich dokumentiert um dritten einen leichtes und umfassendes Verständnis zu ermöglichen.

In einigen Fällen existieren für eine API mehrere konkurrierende Implementierungen, die ggf. von unterschiedlichen Teams erstellt wurden (Beispielsweise wird die **Java Persistence API** implementiert von *Hibernate*, *EclipseLink*, *DataNucleus*, *ObjectDB*...).

Beispiel der Verwendung von Interfaces bei der **Duck**-Klasse:

```
// Das Interface "Omnivore" definiert
// die Methode eat(Eatable).
// Jede Klasse, welche dieses Interface
// implementiert muss auch
// diese Methode implementieren.
public interface Omnivore {
    void eat(Eatable food);
}
```

```
// Das Interface "Flyable" definiert
// die Methode fly().
// Jede Klasse, welche dieses Interface
// implementiert muss auch
// diese Methode implementieren.
public interface Flyable {
    void fly();
}
```

```
// Das Interface "CarnivoreEatable"
// ist ein Marker-Interface mit welchem
// Tiere markiert werden können, welche
// für Fleischfresser als Nahrung
// in Frage kommen.
// Nicht alle Tiere sind essbar
// (z.B. giftig), daher wird diese Markierung
// für jedes Tier individuell vorgenommen.
// Alles was für Fleischfresser essbar ist,
// ist auch für Allesfresser essbar.
// Daher erbt dieses Interface von Eatable.
public interface CarnivoreEatable
    extends Eatable {
}
```

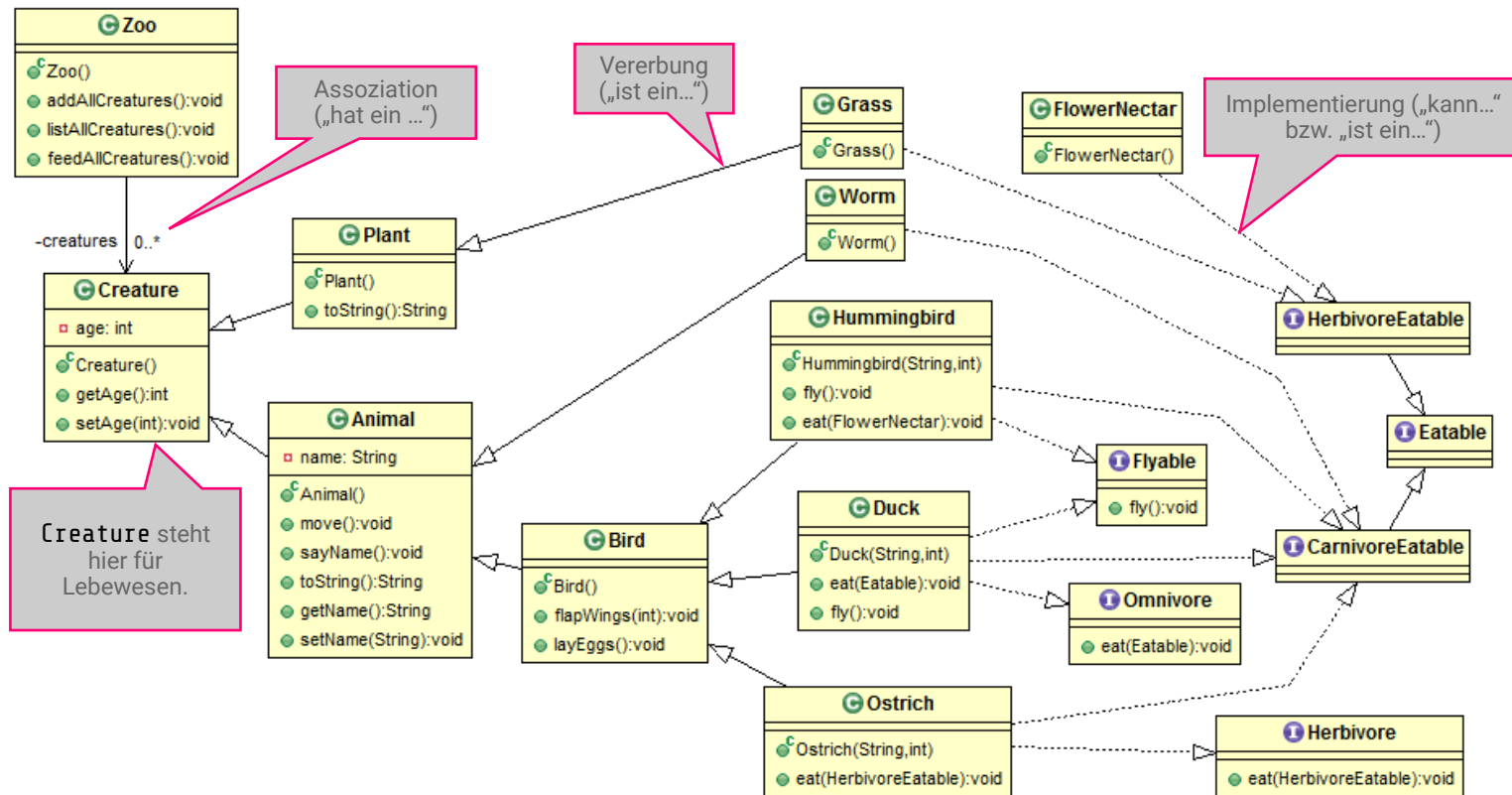
```
// Die Klasse Duck erbt von der Mutterklasse Bird
// und implementiert zusätzlich noch die Interfaces
// Flyable, Omnivore und CarnivoreEatable.
// Das heißt konkret:
// - alle Ducks können fliegen
// - alle Ducks können alles essbare essen
// - alle Ducks sind von Carnivoren/Omnivoren essbar
public class Duck extends Bird
    implements Flyable, Omnivore, CarnivoreEatable {

    // Da die Klasse Duck das Interface Omnivore
    // implementiert, muss Duck auch die Methode
    // eat(Eatable) implementieren.
    public void eat(Eatable food) {
        System.out.println("Eating: " + food);
    }

    // Das Interface "Flyable" verlangt
    // nach der Implementierung von fly().
    public void fly() {
        System.out.println("Flying");
    }

    public Duck(String name, int age) {
        setName(name);
        setAge(age);
    }
}
```

Beispiel eines **Zoos** welcher die Ernährungsgewohnheiten der **Animals** berücksichtigt:



Interfaces und Polymorphie

Der Typ eines Objekts wird durch dessen konkrete Klasse sowie die Interfaces bestimmt, welche durch die konkrete Klasse implementiert werden.

Die Polymorphie erstreckt sich nicht nur auf Klassen, sondern schließt auch Interfaces mit ein.

Beispiel (alle untenstehenden Operationen sind so zulässig):

```
// Polymorphie auf Basis
// der Überklassen von "Duck":
Duck duck = new Duck("Freddy", 2);
Bird bird = duck;
Animal animal = duck;
Creature creature = duck;

// Polymorphie auf Basis
// der Interfaces von "Duck":
Flyable flyable = duck;
Omnivore omnivore = duck;
CarnivoreEatable carnivoreEatable = duck;
Eatable eatable = duck;
```

Im **Heap** existiert nur ein einziges Objekt der **konkreten Klasse Duck**.

Impliziter Cast, denn ein **Duck IST EIN Animal**. Die Variable **animal** verweist auf das einzige **Duck**-Objekt.

Impliziter Cast, denn ein **Duck IST EIN Omnivore**. Die Variable **omnivore** verweist auf das einzige **Duck**-Objekt.

Abstrakte Klassen

Da alle Objekte vom Typ **Bird** mit den Flügeln flattern [**flapWings()**] und Eier legen [**LayEggs()**] können und diese Methoden aktuell bei allen Vögeln gleich funktionieren, sind diese nach wie vor an zentraler Stelle in der Klasse **Bird** implementiert.

Dank Polymorphie sind folgende Instanziierungen möglich:

```
Duck aDuck = new Duck("Freddy", 2);
```

Sowohl die Variable als auch das Objekt haben den Typ **Duck**.

```
Bird anotherDuck = new Duck("Frida", 2);
```

EIN Duck IST EIN Bird, daher funktioniert auch diese Zuweisung.

Aber wie sieht es denn mit **Bird**-Objekten aus?

```
Bird bird = new Bird();
```

Das scheint etwas merkwürdig; wie genau soll denn ein **Bird**-Objekt aussehen?

Von einem **Bird** sollte es keine Objekte geben, nur von konkreten Unterklassen, denn ein Vogel ist mehr ein abstraktes Konzept als ein konkretes Tier.

In Java können Klassen, die niemals instanziiert werden sollen, als **abstract** markiert werden:

Syntax:
`public abstract class MyClass`

Ähnlich zu `flapWings()` und `layEggs()` können alle Vögel ein Geräusch von sich geben [`makeNoise()`], doch ähnlich wie mit der abstrakten Klasse `Bird`, kann man die Frage stellen, wie ein generisches Vogelgeräusch klingen mag.

Es ist viel sinnvoller, dass jede konkrete Unterklasse von `Bird` eine eigene `makeNoise()` Methode implementiert. Hierfür bietet Java die Möglichkeit abstrakter Methoden:

Syntax:
`abstract Type myMethod(ParamType param);`

Abstrakte Methoden besitzen keinen Rumpf und ähneln den Definitionen von Methoden in Interfaces. Klassen, welche abstrakte Methoden enthalten, müssen abstrakte Klassen sein. Jede konkrete Unterklasse muss jede abstrakte Methode ihrer Mutterklasse implementieren.

Häufig erbt eine nicht-abstrakte (konkrete) Klasse von einer abstrakten Klasse und implementiert die abstrakten Methoden (falls vorhanden), sodass dann auch unmittelbar ein Objekt der konkreten Klasse erstellt werden kann.

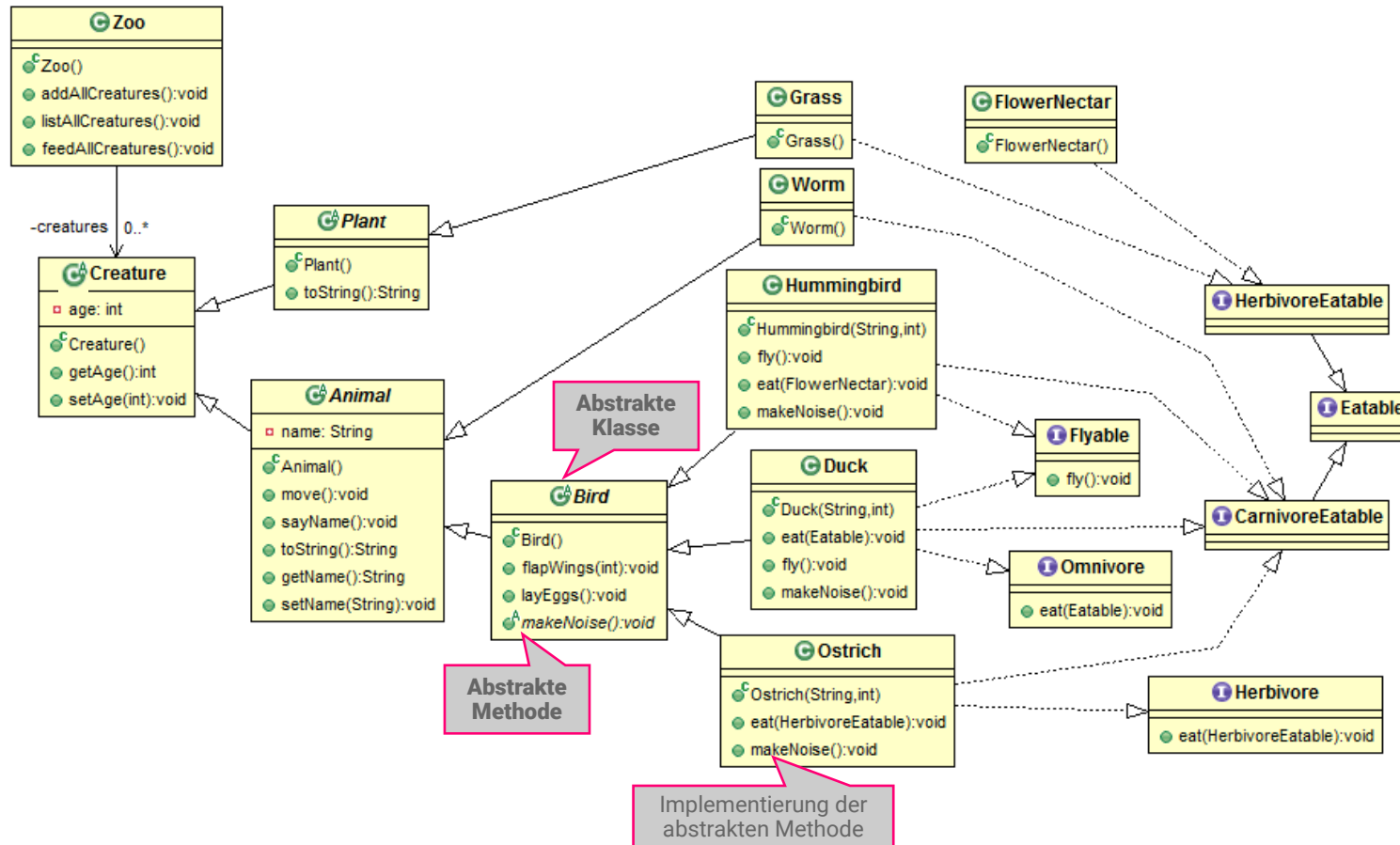
Im Gegensatz zu Interfaces sind innerhalb von abstrakten Klassen auch Instanzvariablen und Methodenrümpfe erlaubt und üblich.

Beispiel des Zusammenspiels zwischen der abstrakten Klasse **Bird** und der konkreten Klasse **Duck**:

```
public abstract class Bird extends Animal {  
    abstract void makeNoise();  
    // es folgen flapWings() und layEggs()  
}
```

```
// Die Klasse Duck erbt von der Mutterklasse Bird  
// und implementiert zusätzlich noch die Interfaces  
// Flyable, Omnivore und CarnivoreEatable.  
// Da die abstrakte Mutterklasse "Bird"  
// die abstrakte Methode "makeNoise()" definiert,  
// muss Duck diese hier konkret implementieren.  
public class Duck extends Bird  
    implements Flyable, Omnivore, CarnivoreEatable {  
    // Konkrete Implementation  
    // der abstrakten Methode von "Bird":  
    public void makeNoise() {  
        System.out.println("Quak, Quak, Quaak!");  
    }  
  
    // Verlangt vom Interface "Omnivore":  
    public void eat(Eatable food) {  
        System.out.println("Eating: " + food);  
    }  
  
    // Verlangt vom Interface "Flyable":  
    public void fly() {  
        System.out.println("Flying");  
    }  
  
    public Duck(String name, int age) {  
        setName(name);  
        setAge(age);  
    }  
}
```

Beispiel des **Zoos** mit Abstrakten Klassen:



Assoziationen und Objekt-Hierarchien

~

Alle **Birds** im **Zoo** können jetzt mit den Flügeln flattern, Geräusche machen, herumlaufen und essen. Außerdem hat jedes **Animal** einen Namen und ein Alter. Um den Fortbestand der verschiedenen Tierarten zu sichern, muss es den Tieren möglich sein, Nachkommen zu zeugen. Dafür muss die bestehende **layEggs()** Methode verbessert werden.

Anforderungen:

- 1 In der Klasse **Bird** soll die neue Instanzvariable **Bird.female (boolean)** das jeweilige Geschlecht angeben. Die zugehörigen Akzessor-Methoden sollen ebenfalls hinzugefügt werden.
- 2 Die Konstruktoren von **Duck**, **Hummingbird** und **Ostrich** sollen so erweitert werden, dass man bei der Objekt-Erzeugung das Geschlecht mit angeben muss.
- 3 In der Klasse **Bird** soll die neue Instanzvariable **Bird.children (Bird[])** hinzugefügt werden. Füge die entsprechende Getter-Methode hinzu. Eine Anpassung des Konstruktors ist nicht nötig.
- 4 Jede konkrete Subklasse von **Bird** muss zwingend eine eigene **layEggs()** Methode implementieren. Sie hat keine Übergabeparameter oder Rückgabewerte.
- 5 Implementiere die Methode **layEggs()** zuerst in der **Duck**-Klasse. Wird die Methode bei der Mutter aufgerufen, werden vier neue Objekte angelegt, die den gleichen Typ wie die Mutter haben (z.B. **Duck**). Jedes Küken hat das Alter 0. Es werden zwei weibliche und zwei männliche Küken angelegt.
- 6 Wird **layEggs()** bei einem männlichen Bird aufgerufen, soll eine Fehlermeldung auf der Konsole ausgegeben werden.
- 7 Wenn ein weiblicher Bird schon einmal zuvor Eier gelegt hat, so kann er nicht nochmal Eier legen.
- 8 Die Instanz-Methode **Bird.listAllChildren()** soll die Infos aller Nachkommen auf der Konsole ausgeben.
- 9 Setze die Methode **layEggs()** auch für alle anderen Unterklassen von **Bird** um.

Der folgende Quellcode setzt alle genannten Anforderungen um:

X

```
public abstract class Bird extends Animal {  
    1 boolean female;  
    Bird[] children; 3  
    abstract void makeNoise();  
    abstract void layEggs(); 4  
    public void listAllChildren() {  
        System.out.println("Children of " + getName() + " :");  
        if (children != null) {  
            8 for (Bird child : children) {  
                System.out.println("- " + child);  
            }  
        }  
        public void flapWings(int count) {  
            for (int i = 0; i < count; i++) {  
                System.out.println("flap");  
            }  
        }  
        3 public Bird[] getChildren() {  
            return children;  
        }  
        1 public boolean isFemale() {  
            return female;  
        }  
        1 public void setFemale(boolean female) {  
            this.female = female;  
        }  
    }  
}
```

```
public class Duck extends Bird implements Flying, Omnivore, CarnivoreEatable {  
    public Duck(String name, int age, boolean female) {  
        setName(name);  
        setAge(age); 2  
        this.female = female;  
    }  
    void layEggs() {  
        if (!isFemale()) { 6  
            System.err.println(getName() + " is male and can't lay eggs!");  
        } else if (getChildren() != null) {  
            7 System.err.println(getName() + " already has children!");  
        } else {  
            children = new Bird[] {  
                new Duck("Mike", 0, false),  
                5 new Duck("Lisa", 0, true),  
                new Duck("Tobi", 0, false),  
                new Duck("Cindy", 0, true),  
            };  
            System.out.println(getName() + " just layed 4 eggs.");  
        }  
    }  
    public void eat(Eatable eatable) {  
        System.out.println(getName() + " is eating: " + eatable);  
    }  
    void makeNoise() {  
        System.out.println(getName() + " says quak!");  
    }  
}
```

Mittels der neuen Methode `Zoo.breedingSeason()` bekommen alle weiblichen Objekte von Typ `Bird` Kinder:

```
public class Zoo {
    private Animal[] allAnimals = new Animal[1000];

    private void addAllAnimals() {
        allAnimals[0] = new Duck("Freddy", 2, false);
        allAnimals[1] = new Duck("Tina", 1, true);
        allAnimals[2] = new Ostrich("Harry", 1, false);
        allAnimals[3] = new Hummingbird("Alice", 2, true);
    }

    public void listAllAnimals() {
        for (Animal currentAnimal : allAnimals) {
            if (currentAnimal != null) {
                System.out.println(currentAnimal.toString());
            }
        }
    }

    public void breedingSeason() {
        System.out.println("Breeding season!");

        for (Animal currentAnimal : allAnimals) {
            if (currentAnimal instanceof Bird) {
                Bird currentBird = (Bird) currentAnimal;

                if (currentBird.isFemale() && currentBird.getChildren() == null) {
                    currentBird.layEggs();

                    for (Bird newbornBird : currentBird.getChildren()) {
                        if (newbornBird != null) {
                            System.out.println(newbornBird.toString());
                        }
                    }
                }
            }
        }
    }
}
```

Der Test-Code:

```
public static void main(String[] args) {
    Zoo myZoo = new Zoo();
    myZoo.addAllAnimals();

    System.out.println("[1]-----");
    myZoo.listAllAnimals();
    System.out.println("[2]-----");
    myZoo.breedingSeason();
}
```

Das Ergebnis:

```
[1]-----
I'm Freddy the Duck and I am 2 years old
I'm Tina the Duck and I am 1 years old
I'm Harry the Ostrich and I am 1 years old
I'm Alice the Hummingbird and I am 2 years old
[2]-----
Breeding season!
Tina just layed 4 eggs.
I'm Mike the Duck and I am 0 years old
I'm Lisa the Duck and I am 0 years old
I'm Tobi the Duck and I am 0 years old
I'm Cindy the Duck and I am 0 years old
Alice just layed 4 eggs.
I'm Ikarus the Hummingbird and I am 0 years old
I'm Maria the Hummingbird and I am 0 years old
I'm Ralf the Hummingbird and I am 0 years old
I'm Helga the Hummingbird and I am 0 years old
```

X

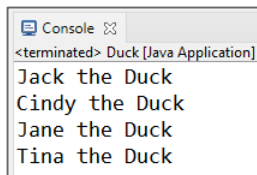
Der Zoo wurde von einer ornithologischen Studie kontaktiert, die sich mit der Erforschung der maternalen mitochondrialen DNA beschäftigt.

Um sich bestmöglich auf die Studienteilnahme vorzubereiten, soll eine einfache Möglichkeit bestehen für jeden **Duck** das Muttertier zu ermitteln. Außerdem soll es einige Möglichkeiten geben Informationen über die Verwandtschaft von verschiedenen Tieren auszugeben bzw. zu ermitteln.

Das sollen folgende Änderungen implementiert werden:

- Es soll ein neues Attribut eingeführt werden, welches eine Referenz auf die jeweilige Mutter jedes **Ducks** enthält
- Eine Methode soll implementiert werden, die ermittelt, ob zwei gegebene **Ducks** miteinander verwandt sind (sofern bekannt)
- Die Ahnenlinie aller Mütter soll mit einer Methode ausgegeben werden (älteste Mutter zuletzt)

```
public static void main(String[] args) {  
    Duck duck = new Duck("Jack", false, new Duck("Cindy", new Duck("Jane", new Duck("Tina"))));  
    duck.printAncestors();  
}
```



Console [x]
<terminated> Duck [Java Application]
Jack the Duck
Cindy the Duck
Jane the Duck
Tina the Duck

Rekursion

Da das Muttertier eines **Duck** Objektes ebenfalls den Typen **Duck** hat, verweist das Feld **Duck.mother** gewissermaßen auf sich selbst. Man spricht dabei von einem **rekursiven Datentyp**, was ein Datentyp ist, der eine Referenz auf sich selbst hält.

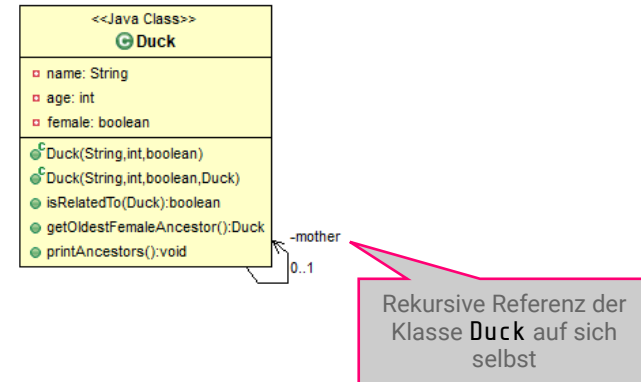
Mit der Methode **Duck.getOldestFemaleAncestor()** soll dasjenige **Duck** Objekt gefunden werden, das an oberster Stelle im Ahnenbaum steht (der ältesten Generation angehört).

Für diese Problemstellung bietet sich die Implementierung eines **rekursiven Algorithmus** an:

- Bei einem rekursiven Algorithmus ruft eine Methode sich selbst auf (entweder direkt oder indirekt mit ein paar Zwischenschritten).
- Diese Kette an Selbst-Aufrufen wird unterbrochen, wenn eine definierte **Abbruchbedingung** erfüllt ist.

Zur Lösung dieses Problems sind lediglich die folgenden Zeilen Code nötig, welche einen rekursiven Algorithmus umsetzen:

```
public Duck getOldestFemaleAncestor() {  
    if (mother == null) {  
        return this;  
    } else {  
        return mother.getOldestFemaleAncestor();  
    }  
}
```

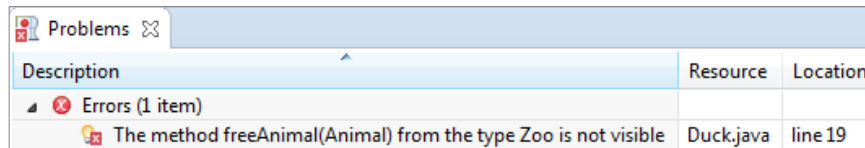


Exceptions

Bei der Softwareentwicklung sind Fehlermeldungen, Abstürze und unerwartetes Programmverhalten alltäglich und absolut normal. Man unterscheidet in Java grundsätzlich zwischen Fehlern, die während der Kompilation auftreten (**Compile-Time**), und solchen, die zur Laufzeit passieren (**Runtime**).

Im Allgemeinen kann eine hundertprozentig fehlerfreie Software nicht garantiert werden. Es kann jedoch mittels verschiedener Arten von Tests sichergestellt werden, dass die üblichsten Funktionen und Konstellationen wie erwartet funktionieren (hierzu später mehr).

Fehler zur Compile-Time sind nicht zu übersehen – schließlich kompiliert der Code gar nicht – und mit Erfahrung meist relativ schnell behebbar:



The screenshot shows the 'Problems' window in the Eclipse IDE. It contains a table with three columns: 'Description', 'Resource', and 'Location'. Under the 'Description' column, there is a red error icon and the text 'Errors (1 item)'. Below that, there is a yellow warning icon and the message 'The method freeAnimal(Animal) from the type Zoo is not visible'. The 'Resource' column shows 'Duck.java' and the 'Location' column shows 'line 19'.

Description	Resource	Location
Errors (1 item)		
The method freeAnimal(Animal) from the type Zoo is not visible	Duck.java	line 19

Ein anderer Teil der Fehler fällt erst zur Laufzeit auf, d.h. wenn der entsprechende Code-Abschnitt tatsächlich ausgeführt wird. In vielen Fällen hängt auch von den verarbeiteten Daten (z.B. fehlerhafte Eingabe) oder Umgebungsfaktoren (z.B. Festplatte voll) ab, ob ein Fehler auftritt.

In Java werden Laufzeitfehler über **Exceptions** angezeigt. Wenn ein Programmbestandteil oder die Java-Laufzeitumgebung einen Fehler feststellt, wird ein neues Objekt vom Typ **Exception** erzeugt, welches Details über den Fehler enthält:

Stacktrace die Information über die Klasse, Methode und Code-Zeile in welcher der Fehler auftrat

Message kurze verbale Beschreibung des Fehlers

Es gibt eine Vielzahl von verschiedenen Exceptions, die in der Regel Unterklassen von **Exception** sind, z.B. **NullPointerException**, **FileNotFoundException** oder **IOException**.

Abfangen von Exceptions

Manchmal kann im Code ein gewisses Risiko nicht vermieden, eine erfolgreiche Ausführung kann nicht garantiert werden. In vielen solchen Fällen, muss man dem Compiler mitteilen, dass man sich das Risiko kennt und sich dessen bewusst ist – ansonsten kann das Programm gar nicht kompiliert werden.

Dafür ist der übliche Weg explizit eine Behandlung für Exceptions zu implementieren, was ein `try/catch`-Konstrukt verwendet. Hierbei kommen in Java drei verschiedene Keywords zum Einsatz:

- try** Markiert einen Code-Block, welcher Code enthält, in welchem Exceptions auftreten können. Falls im `try`-Block eine Exception auftritt, wird die Ausführung an der entsprechenden Zeile abgebrochen und der zum Exception-Typ gehörende Catch-Block aufgerufen.
- catch** Markiert einen Code-Block, in welchem aufgetretene Exceptions behandelt werden. Hier kann z.B. der Fehler geloggt, fehlerhafte Daten korrigiert und/oder eine anwendungsfreundliche Fehlermeldung erzeugt werden. Zu jedem `try`-Block gehört mindestens ein `catch`-Block. Zu einem `try`-Block können optional auch mehrere `catch`-Blocks angelegt werden, die jeweils verschiedene Typen (Klassen) von Exceptions behandeln. Es ist möglich, dass im `try`-Block eine Exception auftritt, welche von keinem `catch`-Block abgefangen wird. In diesem Fall wird die Exception an die aufrufende Methode übergeben.
- finally** Markiert einen optionalen Code-Block der immer nach den `try`- und `catch`-Blöcken ausgeführt wird, egal ob eine Exception aufgetreten ist oder nicht. Hier werden üblicherweise Aufräumarbeiten durchgeführt, z.B. externe Ressourcen wie Datenbankverbindungen geschlossen oder temporäre Dateien entfernt.

Syntax:

```
try {  
    // dangerous code  
} catch (FileNotFoundException exception) {  
    exception.printStackTrace();  
} catch (InterruptedException otherException) {  
    System.err.println(otherException.getMessage());  
} finally {  
    // clean up  
}
```

Werfen von Exceptions

Bevor Exceptions abgefangen und behandelt werden können, müssen sie zuerst erzeugt und geworfen werden. Das passiert innerhalb einer riskanten Methode. Damit die Außenwelt davon erfährt, muss die Methode deklarieren, dass sie möglicherweise eine oder mehrere Exceptions wirft. Dafür wird das Keyword **throws** verwendet:

```
Syntax:  
public void myDangerousMethod() throws IOException, InterruptedException
```

Um im Fehlerfall tatsächlich eine Exception zu werfen, wird zunächst ein Objekt von Typ **Exception** benötigt. Dieses kann entweder mittels **new** neu erzeugt werden oder existiert bereits – etwa durch weiteren Code.

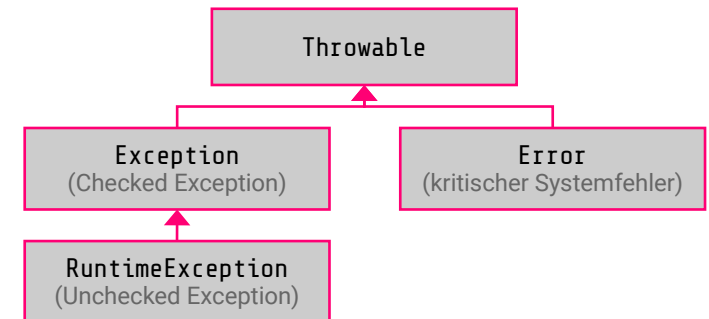
```
Syntax:  
throw new MyException("MyMessage");
```

```
Syntax:  
throw myException;
```

Zusätzlich zur Throws-Klausel sollte immer auch in den Kommentaren (JavaDoc) beschrieben werden, in welchen Situationen eine Methode welche Art von Exceptions wirft.

In Java wird zwischen verschiedenen Grundtypen von Fehlern unterschieden, von denen es jeweils eine große Anzahl an weiteren Subklassen gibt, mit Ausnahme von Throwable:

Exception	Zeigt einen Fehler an, welcher aber kein kritischer Systemfehler ist. Die meisten Fehler sind Subklassen von Exception. Subklassen von Exception sind Checked Exceptions .
RuntimeException	Zeigt einen Fehler an, welcher durch die falsche Verwendung einer API, d.h. typischerweise einen Programmierfehler entstanden ist. Subklassen von RuntimeException sind Unchecked Exceptions .
Error	Zeigt einen kritischen Systemfehler an, z.B. einen vollen Arbeitsspeicher. Einfache Programme sollten keine Errors erzeugen oder catchen.
Throwable	Superklasse aller Fehler.



Checked vs Unchecked Exceptions

Wenn eine Methode deklariert, dass sie eine **Checked Exception** werfen könnte, überprüft der Compiler alle Aufrufe dieser gefährlichen Methode. Sollte sich nicht adäquat um eine eventuell auftretende Exception gekümmert werden, kompiliert der Code nicht. Man kann sich auf zwei mögliche Weisen um eine Exception kümmern:

- Die Checked Exception wird innerhalb der Methode mittels eines **try/catch**-Konstrukts behandelt. In manchen Fällen ist dies aber nicht sinnvoll, weil der Fehler besser in einer der aufrufenden Methoden behandelt werden sollte.
- Die Checked Exception wird nicht innerhalb der ursprünglichen Methode behandelt, sondern wird an die aufrufende Methode weitergereicht (*weitergeworfen*).

Eine Exception weiterzuwerfen bedeutet das Unausweichliche aufzuschieben. Wenn die aufrufende Methode ihrerseits die Exception weiterwirft und sich das bis zur Main-Methode weiterzieht, kommt es zu einem Programmabsturz.

```
Exception in thread "main" java.lang.IndexOutOfBoundsException: Index: 1, Size: 0
    at java.util.ArrayList.rangeCheck(Unknown Source)
    at java.util.ArrayList.remove(Unknown Source)
    at zoo17.company.Zoo.thisWillCrash(Zoo.java:36)
    at zoo17.company.ZooTest.main(ZooTest.java:9)
```

Unchecked Exceptions können auch auf die oben genannten Weisen behandelt werden, allerdings ist dies nicht verpflichtend, da **RuntimeExceptions** zumeist auf logische Fehler in der Programmierung zurückzuführen sind.

Wenn Exceptions auf niedrigen Ebenen komplett abgefangen werden, bekommen höhere Programmkomponenten oder Anwendende ggf. nicht mit, dass überhaupt ein Fehler aufgetreten ist. Es besteht daher die realistische Gefahr, dass Exceptions „versteckt“ werden und dem Anwender ein Erfolg suggeriert wird, obwohl eigentlich ein Fehler aufgetreten ist.

Falls also eine Methode einen Fehler nicht komplett eigenständig lösen kann, ist es häufig ratsam die Exception an die aufrufende Methode weiterzureichen.

Kapitel 3 - Zusammenfassung

- **Arrays** sind Container in welchen mehrere Objekte desselben Typs gespeichert werden können.
- **Polymorphie** erhöht die Flexibilität des Codes, da bei Variablen, Übergabeparametern und Rückgabetypen neben den angegebenen Typen auch beliebige Subtypen verwendet werden können.
- **Interfaces** definieren eine Reihe von Methodensignaturen und geben damit an, welche Funktionalitäten eine Klasse anbietet ohne jedoch eine Implementierung zu beinhalten.
- **Abstrakte Klassen** können ebenfalls un-implementierte Methodensignaturen enthalten, jedoch wie normale Klassen auch Variablen und komplette Methoden.
- Datentypen, die sich selbst oder einen Supertypen von sich als eine Referenz halten (ein Attribut diesen Typs hat), werden als **rekursive Datentypen** bezeichnet.
- Bei einem **rekursiven Algorithmus** ruft sich eine Methode immer wieder selbst auf, bis eine definierte Abbruchbedingung erfüllt ist.
- **Laufzeitfehler (Exceptions)** können direkt behandelt (**try/catch/finally**) oder an die aufrufende Methode weitergereicht werden (oder beides).